

2.9 Paper Organization

In this section, we've laid the groundwork by defining knowledge graph reasoning and discussing the related background knowledge. Moving forward, we'll delve into three distinct perspectives: reasoning for single-hop queries, complex logical queries, and natural language questions. Each perspective offers valuable insights into how knowledge graph reasoning operates and evolves in different contexts. By examining these perspectives, we aim to provide a comprehensive understanding of the diverse challenges and advancements within the field of knowledge graph reasoning. The taxonomy of this paper is illustrated in Figure 1.

3 REASONING FOR SINGLE HOP QUERY

Reasoning for single-hop queries is a common task in the field of knowledge graphs, often referred to as knowledge graph completion. The objective here is to predict the tail entity t given the head entity h and the relation r , or conversely, to predict the head entity h given the tail entity t and the relation r . In addition to entity prediction, there is the relation prediction task, where the goal is to predict the relationship r between a given head entity h and a tail entity t . This task can also be considered a specialized form of single-hop query.

A variety of methods have been proposed to address these tasks. In this section, we categorize the different approaches into three main types: symbolic, neural, and neural-symbolic methods, which will be elaborated in the following subsections.

3.1 Symbolic Methods

3.1.1 Hard symbolic rule based reasoning. Symbolic rule reasoning methods rely on logical reasoning and explicit rules within the knowledge graph. They often utilize rule-based or path-based inference techniques to deduce the missing entity or relation. Examples include rule mining algorithms and path ranking methods and so on.

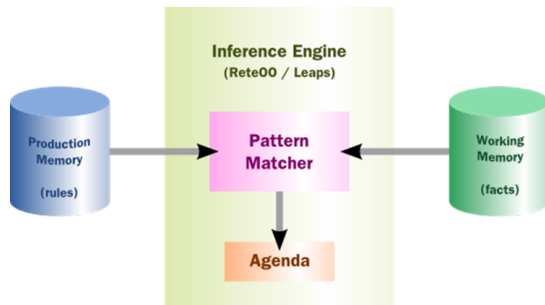


Figure 2: Rule based expert system.

One of the earliest symbolic rule reasoning methods can be traced back to 1970s, which is the rule-based expert system [1] as shown in Figure 2. The key idea of rule-based expert system is to apply hard rules iteratively to generate new facts. It is a very straightforward method. Generally, there are three primary components in rule-based expert system: the inference engine, the knowledge base and the rules defined by experts. The inference engine can be treated as the brain of the reasoning system. It utilizes two methods to infer new knowledge according to the rules. The first method is called forward chaining. The idea is to start with facts and repeatedly

apply the given rules to generate new facts, until no more rules can be applied. The second method is called backward chaining. It is goal oriented, where a goal usually refers to the query from the users. The backward chaining approach will apply the given rules by matching the goal to infer the answer. Besides rule-based expert system,

Alongside rule-based expert systems, Prolog [10] serves as a logic programming language adept at symbolic reasoning through facts and rules, commonly employed in AI applications for knowledge graph querying and manipulation. Similarly, Datalog [8] functions as a declarative logic programming language extensively used for knowledge graph reasoning. It excels in representing complex relationships and hierarchical structures succinctly, enabling tasks such as inference and consistency checking within knowledge graphs. Its logical foundations offer a robust framework for extracting meaningful insights from interconnected data.

Lastly, OWL (Web Ontology Language) [38] is a formal language for defining and sharing ontologies within knowledge graphs on the Semantic Web. OWL enables detailed descriptions of entity classes, properties, and relationships, supporting automated reasoning to ensure data consistency and classification. By providing a standard framework, OWL facilitates interoperability and data integration across systems, enhancing the semantic richness and utility of knowledge graphs.

3.1.2 Soft symbolic rule based reasoning. Despite the idea of hard rule-based reasoning is quite intuitive, it's not the best solution most of the time. Because it requires experts to build rules based on their past experiences and intuitions. So, sometimes, mistakes may be made. Besides, the reasoning method can be very slow because it adds new facts to the knowledge base repeatedly. More importantly, it is not suitable for real time applications. It also lacks flexibility. For example, The reason process will give a world zero probability even if it only violates one formula. But this is not the real-world case. One example is "smoking causes cancer and friends have similar smoking habits". These two rules are not always true, because not everyone who smokes gets cancer. And not all friends have similar smoking habits. And soft rules are more useful in this case because if a word violates a formula, it becomes less probable, not impossible.

1.5	$\forall X \text{ Smokes}(X) \rightarrow \text{Cancer}(X)$
1.1	$\forall X, Y \text{ Friends}(X, Y) \wedge \text{Smokes}(X) \rightarrow \text{Smokes}(Y)$

Two constants: Ana (A) and Bob (B)

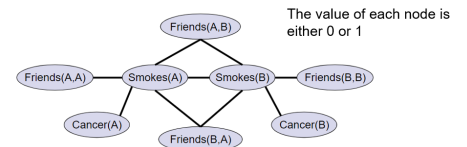


Figure 3: Example of Markov logic network.

Symbolic reasoning methods like Markov logic network [48] is a representative work to reason based on soft rules. The intuition of Markov Logic Network is to give each formula an associated weight to reflect how strong a constraint it is. If the weight is higher, it's a strong constraint, otherwise, it's a weak constraint. When the weights go to infinite, it becomes hard rule reasoning. Formally, Markov network is an undirected graphical model with

Node represent variables. And a potential function is defined for each clique in the graph. The distribution of the Markov network is defined as the multiplication of all the potential functions. When applying Markov network to soft rule reasoning, the idea is to treat the first order logic rules as the templates of Markov network, and reason according to their weights. For example, “smoking causes cancer and friends have similar smoking habits” are two formulas, and they have weight 1.5 and 1.1 respectively, as shown in Figure 3.

Different from Markov logic networks which repeatedly use existing rules to infer knowledge, TensorLog [11] aims to find answers for a given query by matrix multiplication. The idea of Tensorlog is to formulate the reasoning process as a function and represent each entity as a one-hot vector. When applying the function to the input vector, the result is an n by 1 vector where the i th element denotes the probability that entity i is the answer.

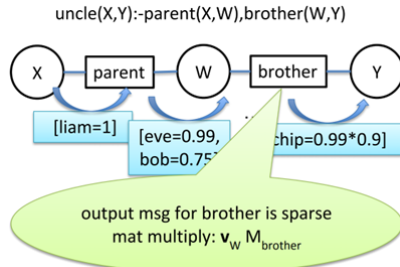


Figure 4: Example of Tensorlog.

An example of Tensorlog is inferring family relations like “uncle” in Figure 4. Given the logic rule $\text{parent}(X, W) \wedge \text{brother}(W, Y) \rightarrow \text{uncle}(X, Y)$, Tensorlog takes the vector of X and multiplies it by the matrix of parent. Then we get W . We multiply the matrix of brother and get the output Y . When the length of the rule becomes longer, and if there are multiple rules, the result is the summation of all the matrix multiplication sequences, and each matrix multiplication sequence has a weight a .

3.1.3 symbolic path based reasoning. For the rule based reasoning methods, they require the rule to be given. However, it’s not the case usually. We don’t have any rules. One possible solution is path-based reasoning. For the path based reasoning method, no rule is needed. It utilizes different paths in the knowledge graph to infer new information. These paths can be directed or undirected.

The very first method of path reasoning is path ranking algorithm [28] which treats the random walks around a query node as the relational features. The idea of PRA is to use random walk to find many different paths between two nodes and treat these paths as the feature of the relation, and use a logistic regression model to learn a classifier to predict the truthfulness of the triplet.

After PRA, ProPPR [19] incorporates vector similarity into random walk inference over KGs to mitigate the feature sparsity inherent in using surface text. Specifically, when conducting a series of edge traversals in a random walk, ProPPR allows the walk to explore edges that exhibit semantic similarity to the given edge types, as defined by vector space embeddings of the edge types. This integration of distributional similarity and symbolic logical inference aims to alleviate the sparsity of the feature space constructed by PRA.

3.1.4 symbolic rule mining. Rule mining can also be treated as a special type of single hop query answering. Instead of directly answer which entity might be the correct answer, Rule mining aims at deducing general logic rules from the knowledge graphs. The entities derived from the given head entity and the query relation following the logic rules are returned as the answers.

AMIE [18] delves into logic rule exploration through a two-step process. Initially, it engages in Rule Extending, wherein candidate rules undergo expansion via three distinct operations: Add Dangling Atom, Add Instantiated Atom and Add Closing Atom. Subsequently, in the Rule Pruning phase, it sifts through the rules, discarding those deemed faulty, and selects the confident ones based on predefined evaluation metrics. In terms of implementation, the approach leverages SPARQL queries on graph databases to sift through the knowledge graphs (KGs), identifying suitable facts (h, r, t) that adhere to the extended rules from the first step and surpass the specified metric thresholds from the second step.

After AMIE, the subsequent algorithm, AMIE+ [17], enhances the efficiency of AMIE’s implementation through adjustments to both the Rule Extending process and the evaluation metrics in the Rule Pruning phase. In the Rule Extending stage, AMIE+ selectively extends a rule only if it can be completed before reaching the predefined maximum rule length. To elaborate, it refrains from appending dangling atoms in the final step, which would introduce fresh variables and lead to non-closure. Instead, AMIE+ waits until the last step to incorporate instantiated atoms and closing atoms, thereby ensuring rule closure. Additionally, the SPARQL queries employed for rule search are streamlined. For instance, when appending a dangling atom to a parent rule R_p to generate a child rule R_c , if the predicate of the new atom already exists in R_p , the support for R_c remains the same as that of R_p . Consequently, recalculating support for R_c becomes unnecessary, thus expediting the SPARQL query process.

While AMIE and AMIE+ have found extensive use across various scenarios, they still face scalability challenges when dealing with large knowledge graphs (KGs). This limitation stems from their reliance on projection queries executed via SQL or SPARQL, where reducing the vast search space remains challenging. In response, RLvLR [43] employs an embedding technique to sample relevant entities and facts pertaining to the target predicate/relation, significantly curtailing the search space. Firstly, RLvLR samples a sub-knowledge graph relevant to the target predicate. Secondly, it utilizes the RESCAL knowledge graph embedding model [42] to generate embeddings for entities and relations in the subgraph, with the embedding for a predicate augmentation being the average value of associated entity embeddings. Thirdly, RLvLR employs a scoring function based on these embeddings to guide and prune rule search, proving effective in rule extraction. Finally, candidate rules are evaluated based on metrics such as *hc* and *conf*, akin to AMIE, computed efficiently through matrix multiplication. By incorporating the embedding technique, RLvLR significantly enhances the efficiency of the rule search process. Another method RuLES [23] utilizes the embedding technique to assess the quality of learned rules. It incorporates external text information of entities to derive their embeddings, enabling the calculation of confidence scores for facts. RuLES defines the external quality of a learned rule as the average confidence score of all derived facts. Ultimately, RuLES

integrates statistical and embedding measures to more precisely evaluate the quality of learned rules.

3.1.5 Summary. In this section, we explore various methods relevant to symbolic reasoning. We discuss rule-based approaches, which leverage logical rules for inference, as well as path-based methods, which analyze patterns within knowledge graphs. Additionally, we delve into rule mining techniques, which aim to extract logical rules from structured data sources. Each method offers unique insights and capabilities in the realm of symbolic reasoning.

3.2 Neural-Symbolic Methods

Neural-symbolic methods aim to combine the strengths of both symbolic and neural methods. They often incorporate symbolic reasoning within a neural framework or use neural networks to enhance symbolic inference.

3.2.1 Knowledge graph embedding. Knowledge graph embedding usually encodes entities as low-dimensional vectors and encodes relations as parametric algebraic operations in the continuous space. The basic idea is to design a score function f which takes the triplet embedding as input, so that a true triplet receives a higher score than a false one. There are a lot of applications which utilize knowledge graph embedding. One of the most common applications is knowledge graph completion. For example, given a head entity and a tail entity, the missing relation is the one which maximizes the score function value. Likewise, given a head entity and a relation, the missing tail entity is the one which, again, maximizes the score function value.

Many KG embedding methods have been developed. The basic idea of TransE [6] is to view the relation r as the transition from the head entity to the tail entity. Mathematically, it means that ideally, the tail entity t should be the summation of the head entity and the relation. Another method is DistMult [70]. Similar as TransE, DistMult also embed entities and relations into vectors in the real/encludence space. Different from TransE, DistMult views the relation r as the elementwise weights of the head entity h . Its score function is defined as the weighted sum over all elements of the head entity by the corresponding elements in the relation. So, in DistMult, the ideal tail entity should be $h\bar{r}$. Another method ComplEx [57] embeds entities and relations in Complex vector space. Each embedding now has a real part and an imaginary part. Given a point z which is $x + iy$ in the embedding space, its conjugate $\bar{z}$ is $x - iy$. The scoring function used in complex is very similar to that of distmult. But we replace t by its conjugate we only taking the real part of the function. Different from dot product, in ComplEx [57] Hermitian dot product $\langle h, r, \bar{t} \rangle$ is asymmetric, where $\bar{t}$ is the complex conjugate of t . Thus it naturally captures the anti-symmetry. Another method is called RotatE [54]. The key idea of rotatE is to solve the limitations in previous methods. similar to complex, rotatE also represent head and tail entities and relation in complex vector space. Different from complex, all the relation embedding are modelled as rotation from the head entity h to the tail entity t . Compared with other methods, RotatE can support different relation properties, such as symmetry/antisymmetry, inversion, and composition. Other methods such as TransH

[67] embeds knowledge graph into the hyperplane of a specific relationship to measure the distance. TransR [31] represents entities and relationships in separate entity and relationship spaces. The semantic matching model uses semantic similarity to score the relationship between head entities and tail entities. RESCAL [42] treats each entity as a vector to capture its implied semantics and uses the relationship matrix to model the interactions between latent factors. QuatE [80] uses two rotating planes to model the relations to a hyper-complex space. HolE [41] employs cyclic correlation to represent the composition of the graph. However, neither of these methods captures the structure information of the graph which should be important to the graph.

In addition to point embeddings, recent methods have explored using geometric regions to represent knowledge graphs in embedding spaces. Geometric embedding techniques include regions like boxes and spheres, which are effective in modeling relationships and hierarchical structures within knowledge graphs [15, 59]. Other approaches employ probabilistic embeddings to represent knowledge graphs. For instance, probability-based embeddings, such as Gaussian distributions, capture uncertainty and variability in the data, providing a probabilistic interpretation of the embeddings [22, 60]. These methods enhance the expressiveness and flexibility of embeddings, enabling more robust reasoning and inference in various applications.

3.2.2 Neural symbolic rule based reasoning. Neural LP [72], which is a generalization of Tensorlog that focuses on learning logical rules with confidence scores. In Tensorlog, the reasoning process is a sequence of matrix multiplication operations. Tensorlog denotes the input query entity as a one-hot vector and each relation as a matrix R . The reasoning results are computed by matrix multiplication, retrieving entities whose entries are non-zero as answers. Neural LP adopts the same idea as Tensorlog. In Neural LP, the authors found that when the rule length increases from 2 to L , the original first matrix multiplication then summation process can be rewritten as the first summation then multiplication process. After changing the order of the operations, the original matrix multiplication process becomes learning the combination of relationships at each step. This process can be modeled by a recurrent neural network (RNN) for T steps. The candidate pool of Neural LP is very large, which leads to a huge search space. So, it's hard to identify useful rules in the search space. Most of the time the weights may not reflect the importance of rules precisely. To solve these limitations, RNNLogic [44] treats all logic rules as latent variables. That is, to answer a query, there may be more than 10 or 20 related rules, and we treat all these logic rules as latent variables. In this way, the rule mining problem becomes a rule inference problem. RNNLogic contains two components: the rule generator, which will generate some candidate logic rules for a specific query, and a reasoning predictor, which is used to predict how likely we can find the answer given a logic rule. Different from Neural LP, the search space of RNNLogic is much smaller. Because all logic rules are treated as latent variables, the EM algorithm can be used for inference.

3.2.3 Neural symbolic path based methods. Previously, we introduced symbolic path-based reasoning methods. Neural symbolic